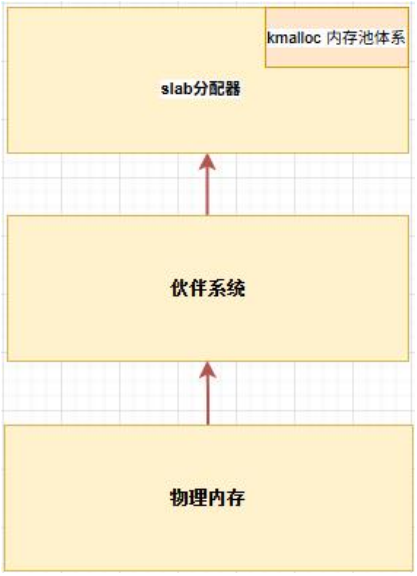
11 Linux 内核级通用内存池 —— kmalloc 体系 2
11.1 kmalloc 内存池中包含的内存块大小 2
11.2 kmalloc 内存池如何选取合适尺寸的内存块 4
11.3 kmalloc 内存池的整体架构 5
11.3.1 kmalloc 内存池尺寸划分逻辑 5
11.3.2 kmalloc 内存池中内存块的所属物理内存区域 6
11.4 kmalloc 内存池的创建 9
11.4.1 创建二维数组 kmalloc_caches 10
11.4.2 创建具体尺寸的 slab cache 12
11.5 kmalloc 内存池内存分配过程 14
11.5.1 从 kmalloc_caches 中选取最佳尺寸的 slab cache 17
11.5.1.2 计算 kmalloc_caches 数组的第一个维度索引的值 18
11.5.1.2 计算 kmalloc_caches 数组的第二个维度索引的值 19
11.6 kmalloc 内存池对内存回收 22

11 Linux 内核级通用内存池 —— kmalloc 体系

kmalloc 内存池体系是为了应对，比如，8 字节，16 字节，32 字节等通用小内存尺寸的频繁分配释放需求的。在内核启动初始化的时候，通过 kmem_cache_create 接口函数预先创建多个特定尺寸的 slab cache 出来，用以应对不同尺寸的通用内存块的申请。其中，size 参数来指定要创建的通用内存块尺寸。

```
struct kmem_cache *
kmem_cache_create(const char *name, unsigned int size, unsigned int align,
        slab_flags_t flags, void (*ctor)(void *))
```

kmalloc 内存池体系的底层基石是基于 slab alloactor 体系构建的，其本质其实就是各种不同尺寸的通用 slab cache。



11.1 kmalloc 内存池中包含的内存块大小

kmalloc 内存池中都有哪些尺寸的内存块？内核将这些不同尺寸的 slab cache 分类信息定义在 kmalloc_info[] 数组中，数组中的元素类型为 kmalloc_info_struct 结构，里边定义了对应尺寸通用内存池的相关信息。

```
/*
 * kmalloc_info[] is to make slub_debug=,kmalloc-xx option work at boot time.
 * kmalloc_index() supports up to 2^21=2MB, so the final entry of the table
is
 * kmalloc-2M.
 */
const struct kmalloc_info_struct kmalloc_info[] __initconst = {
    INIT_KMALLOC_INFO(0, 0),
    INIT_KMALLOC_INFO(96, 96),
```

```
    INIT_KMALLOC_INFO(192, 192),
    INIT_KMALLOC_INFO(8, 8),
    INIT_KMALLOC_INFO(16, 16),
    INIT_KMALLOC_INFO(32, 32),
    INIT_KMALLOC_INFO(64, 64),
    INIT_KMALLOC_INFO(128, 128),
    INIT_KMALLOC_INFO(256, 256),
    INIT_KMALLOC_INFO(512, 512),
    INIT_KMALLOC_INFO(1024, 1k),
    INIT_KMALLOC_INFO(2048, 2k),
    INIT_KMALLOC_INFO(4096, 4k),
    INIT_KMALLOC_INFO(8192, 8k),
    INIT_KMALLOC_INFO(16384, 16k),
    INIT_KMALLOC_INFO(32768, 32k),
    INIT_KMALLOC_INFO(65536, 64k),
    INIT_KMALLOC_INFO(131072, 128k),
    INIT_KMALLOC_INFO(262144, 256k),
    INIT_KMALLOC_INFO(524288, 512k),
    INIT_KMALLOC_INFO(1048576, 1M),
    INIT_KMALLOC_INFO(2097152, 2M)
};
```

```
/* A table of kmalloc cache names and sizes */
extern const struct kmalloc_info_struct {
    // slab cache 的名字
    const char *name[NR_KMALLOC_TYPES];
     // slab cache 提供的内存块大小，单位为字节
    unsigned int size;
} kmalloc_info[];
```

size 用于指定该 slab cache 中所管理的通用内存块尺寸。

name 为该通用 slab cache 的名称，名称形式为 kmalloc-内存块尺寸（单位字节）。

从 kmalloc_info[] 数组中我们可以看出，kmalloc 内存池体系理论上最大可以支持 2M 尺寸大小的通用内存池。kmalloc_info[] 数组中的 index 有一个特点，从 index = 3 开始一直到数组的最后一个 index，这其中的每一个 index 都表示其对应的 kmalloc_info[index] 指向的通用 slab cache 尺寸，也就是说 kmalloc 内存池体系中的每个通用 slab cache 中内存块的尺寸由其所在的 kmalloc_info[] 数组 index 决定，对应内存块大小为：2^index 字节，比如：

kmalloc_info[3] 对应的通用 slab cache 中所管理的内存块尺寸为 8 字节。

kmalloc_info[5] 对应的通用 slab cache 中所管理的内存块尺寸为 32 字节。

kmalloc_info[9] 对应的通用 slab cache 中所管理的内存块尺寸为 512 字节。

kmalloc_info[index] 对应的通用 slab cache 中所管理的内存块尺寸为 2^index 字节。

但是这里的 index = 1 和 index = 2 是个例外，内核单独支持了 kmalloc-96 和 kmalloc-192 这两个通用 slab cache。它们分别管理了 96 字节大小和 192 字节大小的通用内存块。这些内存块的大小都不是 2 的次幂。

在内核中，对于内存块的申请需求大部分情况下都在 96 字节或者 192 字节附近，如果内核不单独支持这两个尺寸的通用 slab cache。那么当内核申请一个尺寸在 64 字节到 96 字节之间的内存块时，内核会直接从 kmalloc-128 中分配一个 128 字节大小的内存块，这样就导致了内存块内部碎片比较大，浪费宝贵的内存资源。因此，内核单独支持了 kmalloc-96 和 kmalloc-192 这两个通用 slab cache。

11.2 kmalloc 内存池如何选取合适尺寸的内存块

内核通过定义一个 size_index[24] 数组来存放申请内存块大小在 192 字节以下的 kmalloc 内存池选取规则。其中 ，size_index[24] 数组中每个元素后面跟的注释部分为内核要申请的字节数，size_index[24] 数组中每个元素表示最佳合适尺寸的通用 slab cache 在 kmalloc_info[] 数组中的索引。

```
/* similar to kmalloc, but sizeof == 8 bucket is gone */
static u8 size_index[24] __ro_after_init = {
    3,  /* 8 */
    3,  /* 16 */
    4,  /* 24 */
    4,  /* 32 */
    5,  /* 40 */
    5,  /* 48 */
    5,  /* 56 */
    5,  /* 64 */
    1,  /* 72 */
    1,  /* 80 */
    1,  /* 88 */
    1,  /* 96 */
    6,  /* 104 */
    6,  /* 112 */
    6,  /* 120 */
    6,  /* 128 */
    2,  /* 136 */
    2,  /* 144 */
    2,  /* 152 */
    2,  /* 160 */
    2,  /* 168 */
    2,  /* 176 */
    2,  /* 184 */
    2   /* 192 */
};
```

size_index[0] 存储的信息表示，如果内核申请的内存块低于 8 字节时，那么 kmalloc 将会到 kmalloc_info[3] 所指定的通用 slab cache —— kmalloc-8 中分配一个 8 字节大小的内存块。

size_index[16] 存储的信息表示，如果内核申请的内存块在 128 字节到 136 字节之间

时，那么 kmalloc 将会到 kmalloc_info[2] 所指定的通用 slab cache —— kmalloc-192 中分配一个 192 字节大小的内存块。

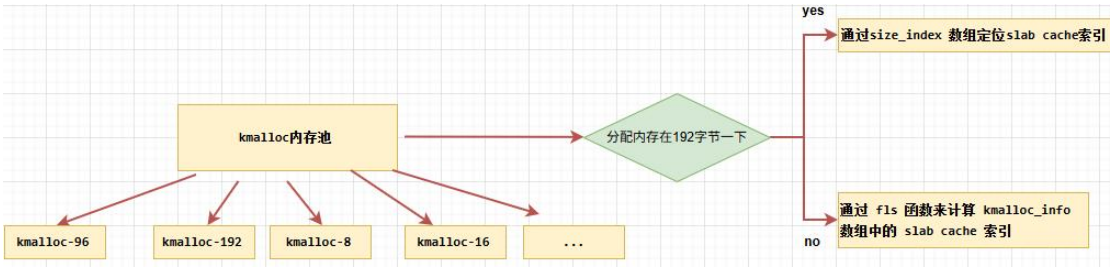
同样的道理，申请 144，152，160 ..... 192 等字节尺寸的内存块对应的最佳 slab cache 选取规则也是如此，都是通过 size_index 数组中的值找到 kmalloc_info 数组的索引，然后通过 kmalloc_info[index] 指定的 slab cache，分配对应尺寸的内存块。

size_index 数组只是定义申请内存块在 192 字节以下的 kmalloc 内存池选取规则，当申请内存块的尺寸超过 192 字节时，内核会通过 fls 函数来计算 kmalloc_info 数组中的通用 slab cache 索引。这一点我们在后续源码分析中还会在提到，这里大家有个大概印象即可。

11.3 kmalloc 内存池的整体架构

11.3.1 kmalloc 内存池尺寸划分逻辑

kmalloc 内存池的本质其实还是 slab 内存池，底层依赖于 slab alloactor 体系，在 kmalloc 体系的内部，管理了多个不同尺寸的 slab cache，kmalloc 只不过负责根据内核申请的内存块尺寸大小来选取一个最佳合适尺寸的 slab cache。最终内存块的分配和释放还需要由底层的 slab cache 来负责.



以 slub 实现来讨论。kmalloc 体系所能支持的内存块尺寸范围由 KMALLOC_SHIFT_LOW 和 KMALLOC_SHIFT_HIGH 决定，它们被定义在 /include/linux/slab.h 文件中：

```
    // slub 最大支持分配 2 页 大小的对象，
    //对应的 kmalloc 内存池中内存块尺寸最大就是 2 页
    // 超过 2 页 大小的内存块直接向伙伴系统申请
/*
 * SLUB directly allocates requests fitting in to an order-1 page
 * (PAGE_SIZE*2).  Larger requests are passed to the page allocator.
 */
  #define KMALLOC_SHIFT_HIGH  (PAGE_SHIFT + 1)
  #define KMALLOC_SHIFT_LOW   3

// arch\x86\include\asm\page_types.h
/* PAGE_SHIFT determines the page size */
#define PAGE_SHIFT      12

// arch\arm64\include\asm\page-def.h
```

```
#define PAGE_SHIFT      CONFIG_ARM64_PAGE_SHIFT
```

其中， kmalloc 支持的最小内存块尺寸为：2^KMALLOC_SHIFT_LOW，在 slub 实现中 KMALLOC_SHIFT_LOW = 3，kmalloc 支持的最小内存块尺寸为 8 字节大小。

kmalloc 支持的最大内存块尺寸为：2^KMALLOC_SHIFT_HIGH，在 slub 实现中 KMALLOC_SHIFT_HIGH = 13，kmalloc 支持的最大内存块尺寸为 8K ，也就是两个内存页大小。所以，在内核的 slub 实现中，kmalloc 所能支持的内存块大小在 8 字节到 8K 之间。

kmalloc 体系中的内存块尺寸划分逻辑如上所述。

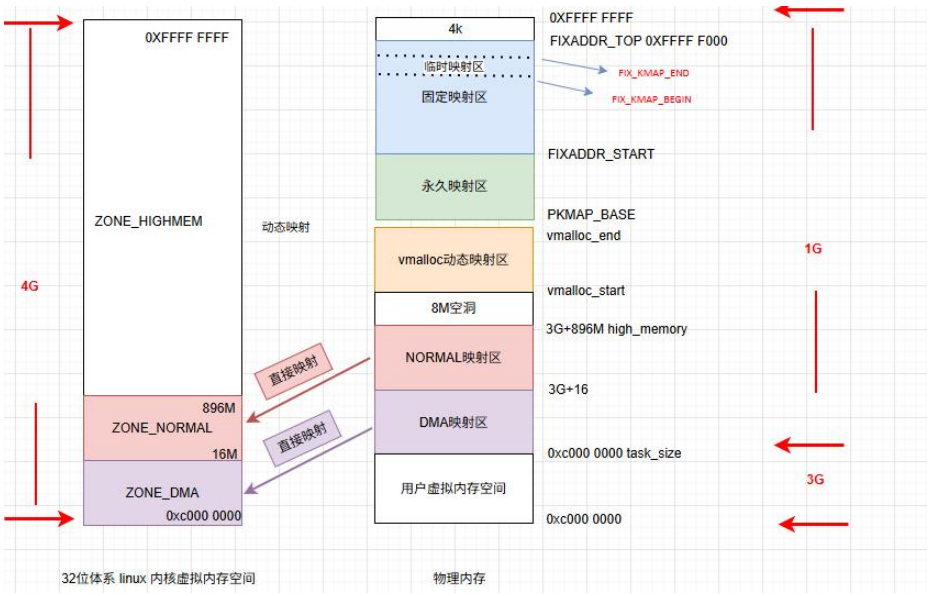
11.3.2 kmalloc 内存池中内存块的所属物理内存区域

内核会根据各个物理内存区域的功能不同，将 NUMA 节点内的物理内存划分为以下几个物理内存区域：

```
enum zone_type {
#ifdef CONFIG_ZONE_DMA
    ZONE_DMA,
#endif
#ifdef CONFIG_ZONE_DMA32
    ZONE_DMA32,
#endif
    ZONE_NORMAL,
#ifdef CONFIG_HIGHMEM
    ZONE_HIGHMEM,
#endif
    ZONE_MOVABLE,
#ifdef CONFIG_ZONE_DEVICE
    ZONE_DEVICE,
#endif
    __MAX_NR_ZONES

};
```

kmalloc 内存池中的内存来自于 ZONE_DMA 和 ZONE_NORMAL 物理内存区域，也就是内核虚拟内存空间中的直接映射区域。



kmalloc 内存池中的内存来源类型定义在 /include/linux/slab.h 文件中：

```
enum kmalloc_cache_type {
   // 规定 kmalloc 内存池的内存需要在 NORMAL 直接映射区分配
    KMALLOC_NORMAL = 0,
#ifndef CONFIG_ZONE_DMA
     //CONFIG_ZONE_DMA 未启用：系统无 DMA 区域需求，
      //无需独立的 DMA 缓存池，KMALLOC_DMA 等价于 KMALLOC_NORMAL
      KMALLOC_DMA = KMALLOC_NORMAL,
#endif
#ifndef CONFIG_MEMCG_KMEM
    KMALLOC_CGROUP = KMALLOC_NORMAL,
#endif
    KMALLOC_RANDOM_START = KMALLOC_NORMAL,
    KMALLOC_RANDOM_END = KMALLOC_RANDOM_START + RANDOM_KMALLOC_CACHES_NR,
#ifdef CONFIG_SLUB_TINY
    KMALLOC_RECLAIM = KMALLOC_NORMAL,
#else
    // 规定 kmalloc 内存池中的内存是可以回收的，比如文件页缓存，匿名页
    KMALLOC_RECLAIM,
#endif
#ifdef CONFIG_ZONE_DMA
    //CONFIG_ZONE_DMA 启用
    // kmalloc 内存池中的内存用于 DMA，需要在 DMA 区域分配
    KMALLOC_DMA,
#endif
#ifdef CONFIG_MEMCG_KMEM
    KMALLOC_CGROUP,
#endif
    NR_KMALLOC_TYPES
```

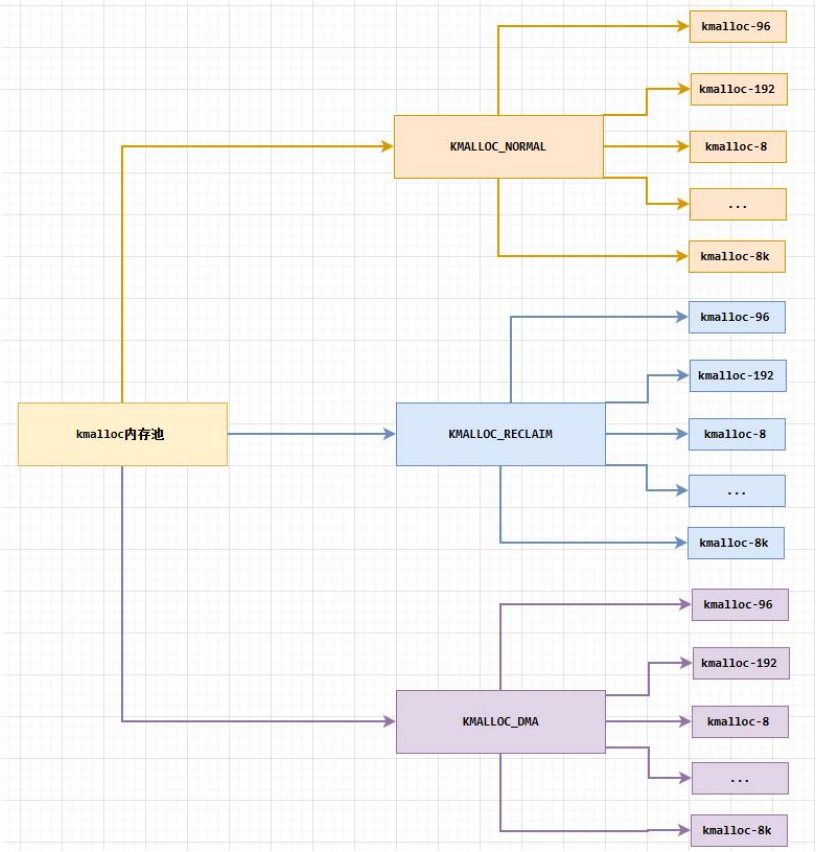
```
};
```

KMALLOC_NORMAL 表示 kmalloc 需要从 ZONE_NORMAL 物理内存区域中分配内存。

KMALLOC_DMA 表示 kmalloc 需要从 ZONE_DMA 物理内存区域中分配内存。

KMALLOC_RECLAIM 表示需要分配可以被回收的内存，RECLAIM 类型的内存页，不能移动，但是可以直接回收，比如文件缓存页，它们就可以直接被回收掉，当再次需要的时候可以从磁盘中读取生成。或者一些生命周期比较短的内存页，比如 DMA 缓存区中的内存页也是可以被直接回收掉。

把 kmalloc 内存池中的内存来源加上，kmalloc 的总体架构如下：



内核将上述 kmalloc 内存池整体架构体系定义在一个 kmalloc_caches 二维数组中，位于文件：/include/linux/slab.h 中。

```
    extern struct kmem_cache *
kmalloc_caches[NR_KMALLOC_TYPES][KMALLOC_SHIFT_HIGH + 1];
```

第一维数组用于表示 kmalloc 内存池中的内存来源于哪些物理内存区域中，也就是前边介绍的 enum kmalloc_cache_type。

第二维数组中的元素一共 KMALLOC_SHIFT_HIGH 个，用于存储每种内存块尺寸对应的 slab cache。在 slub 实现中，kmalloc 内存池中的内存块尺寸在 8 字节到 8K 之间，其中还包括了两个特殊的尺寸分别为 96 字节 和 192 字节。

第二维数组中的 index 表示的含义和 kmalloc_info[] 数组中的 index 含义一模一样，均是表示对应 slab cache 中内存块尺寸的分配阶（2 的次幂）。96 和 192 这两个内存块尺寸除外，它们的 index 分别是 1 和 2，单独特殊指定。

11.4 kmalloc 内存池的创建

kmalloc 体系底层依赖的是 slab allocator 体系，所以 kmalloc 体系的创建是在 slab allocator 体系创建之后进行的。在内核初始化内存管理子系统的时候，会在 kmem_cache_init 函数中完成 slab alloactor 体系的创建初始化工作，之后紧接着就会创建初始化 kmalloc 体系。

```
asmlinkage __visible __init __no_sanitize_address __noreturn
__no_stack_protector
void start_kernel(void)
{
    ........ 省略 .........
     mm_core_init();
    ........ 省略 .........
}
```

```
void __init mm_core_init(void)
{

    ........ 省略 .........
     // 初始化内存管理子系统
    mem_init();
    ........ 省略 .........
    // 创建并初始化 slab allocator 体系
    kmem_cache_init();
     ........ 省略 .........

 }
```

```
void __init kmem_cache_init(void)
{
     ........... 省略 slab allocator 体系的创建初始化过程 ......
     /* Now we can use the kmem_cache to allocate kmalloc slabs */
     // 初始化上边提到的 size_index 数组
    setup_kmalloc_cache_index_table();

    // 创建 kmalloc_info 数组中保存的各个内存块大小对应的 slab cache
    // 最终将这些不同尺寸的 slab cache 缓存在 kmalloc_caches 中
    create_kmalloc_caches(0);

}
```

kmalloc 体系的初始化工作核心分为两个部分：

1. setup_kmalloc_cache_index_table 初始化 size_index 数组，后续 kmalloc 在分配 192 字节以下的内存块时，内核会利用该数组选取最佳合适尺寸的 slab cache。

2. create_kmalloc_caches 创建初始化 kmalloc_caches 二维数组，这个二维数组是 kmalloc 体系的核心。内核会利用 kmalloc_caches 直接找到对应的 slab cache 进行内存块的分配和释放。

11.4.1 创建二维数组 kmalloc_caches

create_kmalloc_caches 函数的主要任务就是创建和初始化 kmalloc_caches 二维数组，它会为每一个 enum kmalloc_cache_type 分别创建 2^KMALLOC_SHIFT_LOW（8 字节） 到 2^KMALLOC_SHIFT_HIGH（8K） 范围内的 slab cache。当然也包括两个特殊的 slab cache 尺寸，他俩分别是：kmalloc-96，kmalloc-192，剩下的 slab cache 尺寸必须是 2 的次幂。

```
extern struct kmem_cache *
kmalloc_caches[NR_KMALLOC_TYPES][KMALLOC_SHIFT_HIGH + 1];
```

```
#define KMALLOC_SHIFT_LOW   3
#define KMALLOC_MIN_SIZE (1 << KMALLOC_SHIFT_LOW)

    /*
 * Create the kmalloc array. Some of the regular kmalloc arrays
 * may already have been created because they were needed to
 * enable allocations for slab creation.
 */
void __init create_kmalloc_caches(slab_flags_t flags)
{
    int i;
    enum kmalloc_cache_type type;
    //初始化二维数组 kmalloc_caches，为每一个 kmalloc_cache_type
    //类型创建内存块尺寸从 KMALLOC_SHIFT_LOW 到
    //KMALLOC_SHIFT_HIGH 大小的 kmalloc 内存池
    for (type = KMALLOC_NORMAL; type < NR_KMALLOC_TYPES; type++) {
     // 从 8 字节 尺寸的内存池开始创建，一直到创建完 8K 尺寸的内存池
        for (i = KMALLOC_SHIFT_LOW; i <= KMALLOC_SHIFT_HIGH; i++) {
            if (!kmalloc_caches[type][i])
            // 创建对应尺寸的 kmalloc 内存池，其中内存块大小为 2^i 字节
                new_kmalloc_cache(i, type, flags);
             // 创建 kmalloc-96 内存池管理 96 字节 尺寸的内存块
            // 专门创建一个 96 字节尺寸的内存池的目的是为了，
           //应对 64B 到 128B 之间的内存分配需求，
           //要不然超过 64B 就分配一个 128B 的内存块有点浪费
            if (KMALLOC_MIN_SIZE <= 32 && i == 6 &&
                    !kmalloc_caches[type][1])
                new_kmalloc_cache(1, type, flags);
            // 创建 kmalloc-192 内存池管理 192B 尺寸的内存块
```

```
            // 这里专门创建一个 192 字节尺寸的内存池.
             //是为了分配 128 字节 到 192 字节 之间的内存分配需求
            // 要不然超过 128 字节 直接分配一个 256 字节 的内存块太浪费了
           if (KMALLOC_MIN_SIZE <= 64 && i == 7 &&
                   !kmalloc_caches[type][2])
               new_kmalloc_cache(2, type, flags);
        }
    }
#ifdef CONFIG_RANDOM_KMALLOC_CACHES
    random_kmalloc_seed = get_random_u64();
#endif
    /* Kmalloc array is now usable */
    // 当 kmalloc 体系全部创建完毕之后，slab 体系的状态就变为 up 状态
    slab_state = UP;
}
```

关于 create_kmalloc_caches 函数的逻辑的几点：

在第一个 for 循环体内的二重循环里，当 i = 6 时，表示现在准备要创建 2^6 = 64 字节尺寸的 slab cache —— kmalloc-64，当创建完 kmalloc-64 时，需要紧接着特殊创建 kmalloc-96，而 kmalloc-96 在 kmalloc_info 数组和 kmalloc_caches 二维数组中的索引均是 1，所以调用 new_kmalloc_cache 创建具体尺寸的 slab cache 时候，第一个参数指的是 slab cache 在 kmalloc_caches 数组中的 index，这里传入的是 1。

同样，在当 i = 7 时，表示现在准备要创建 2^7 = 128 字节尺寸的 slab cache —— kmalloc-128，然后紧接着就需要特殊创建 kmalloc-192，而 kmalloc-192 在 kmalloc_caches 二维数组中的索引是 2，所以 new_kmalloc_cache 第一个参数传入的是 2。

当 KMALLOC_NORMAL 和 KMALLOC_RECLAIM 这两个类型的 kmalloc 内存池建立起来之后，slab_state 就变成了 UP 状态，表示现在 slab allocator 体系已经建立起来了，可以正常运转了。

```
/*
 * State of the slab allocator.
 *
 * This is used to describe the states of the allocator during bootup.
 * Allocators use this to gradually bootstrap themselves. Most allocators
 * have the problem that the structures used for managing slab caches are
 * allocated from slab caches themselves.
 */
enum slab_state {
    DOWN,           /* No slab functionality yet */
    PARTIAL,        /* SLUB: kmem_cache_node available */
    PARTIAL_NODE,       /* SLAB: kmalloc size for node struct available */
    UP,         /* Slab caches usable but not all extras yet */
    FULL            /* Everything is working */
};
```

11.4.2 创建具体尺寸的 slab cache

使用 new_kmalloc_cache 来创建具体尺寸的 slab cache。然后，create_kmalloc_caches 函数又根据 kmalloc_info[ ] 数组中的 index 来创建对应尺寸的 slab cache 。

new_kmalloc_cache 函数源码如下:

```
/*
 * kmalloc_info[] is to make slub_debug=,kmalloc-xx option work at boot time.
 * kmalloc_index() supports up to 2^21=2MB, so the final entry of the table
is
 * kmalloc-2M.
 */
const struct kmalloc_info_struct kmalloc_info[] __initconst = {
    INIT_KMALLOC_INFO(0, 0),
    INIT_KMALLOC_INFO(96, 96),
    INIT_KMALLOC_INFO(192, 192),
    INIT_KMALLOC_INFO(8, 8),
    INIT_KMALLOC_INFO(16, 16),
    INIT_KMALLOC_INFO(32, 32),
    INIT_KMALLOC_INFO(64, 64),
    INIT_KMALLOC_INFO(128, 128),
    INIT_KMALLOC_INFO(256, 256),
    INIT_KMALLOC_INFO(512, 512),
    INIT_KMALLOC_INFO(1024, 1k),
    INIT_KMALLOC_INFO(2048, 2k),
    INIT_KMALLOC_INFO(4096, 4k),
    INIT_KMALLOC_INFO(8192, 8k),
    INIT_KMALLOC_INFO(16384, 16k),
    INIT_KMALLOC_INFO(32768, 32k),
    INIT_KMALLOC_INFO(65536, 64k),
    INIT_KMALLOC_INFO(131072, 128k),
    INIT_KMALLOC_INFO(262144, 256k),
    INIT_KMALLOC_INFO(524288, 512k),
    INIT_KMALLOC_INFO(1048576, 1M),
    INIT_KMALLOC_INFO(2097152, 2M)
};
```

new_kmalloc_cache 函数核心作用是根据缓存池类型（DMA/CGROUP/RECLAIM 等）、内核配置（memcg/DMA/ 随机化等）和内存对齐要求，设置对应的 SLAB 标志位，计算对齐后的内存大小与索引，最后创建或复用 kmalloc 缓存池。源码实现如下：

```
void __init
new_kmalloc_cache(int idx, enum kmalloc_cache_type type, slab_flags_t
flags)
{
```

```
    // kmalloc 分配的最小对齐要求
    unsigned int minalign = __kmalloc_minalign();
    //从 kmalloc_info 数组中获取 idx 对应的原始内存块大小
    unsigned int aligned_size = kmalloc_info[idx].size;
    //初始化对齐后的索引为原始索引
    int aligned_idx = idx;

     //RECLAIM 类型缓存池（可回收对象）
    if ((KMALLOC_RECLAIM != KMALLOC_NORMAL) && (type == KMALLOC_RECLAIM))
{
        //追加 SLAB_RECLAIM_ACCOUNT 标志；
        flags |= SLAB_RECLAIM_ACCOUNT;
    } else if (IS_ENABLED(CONFIG_MEMCG_KMEM) && (type == KMALLOC_CGROUP))
{  //CGROUP 类型缓存池：启用 CONFIG_MEMCG_KMEM，且 type == KMALLOC_CGROUP
        if (mem_cgroup_kmem_disabled()) {
           //若 功能被禁用，则复用 KMALLOC_NORMAL 的缓存池，
          //直接返回（无需创建新缓存池）；
            kmalloc_caches[type][idx] =
kmalloc_caches[KMALLOC_NORMAL][idx];
            return;
        }
        //若 memcg 启用，追加 SLAB_ACCOUNT 标志
        flags |= SLAB_ACCOUNT;
    } else if (IS_ENABLED(CONFIG_ZONE_DMA) && (type == KMALLOC_DMA)) {
    //DMA 类型缓存池。条件：启用 CONFIG_ZONE_DMA 且 type == KMALLOC_DMA；
       //追加 SLAB_CACHE_DMA 标志；
        //强制该缓存池的内存块分配在 ZONE_DMA 区域
         flags |= SLAB_CACHE_DMA;
    }
    //随机化缓存池的合并禁用
    //启用 CONFIG_RANDOM_KMALLOC_CACHES（kmalloc 缓存池地址随机化），
    //且类型小于 KMALLOC_RANDOM_END；
#ifdef CONFIG_RANDOM_KMALLOC_CACHES
    if (type >= KMALLOC_RANDOM_START && type <= KMALLOC_RANDOM_END)
       //追加 SLAB_NO_MERGE 标志
        flags |= SLAB_NO_MERGE;
#endif
    /*
     * If CONFIG_MEMCG_KMEM is enabled, disable cache merging for
     * KMALLOC_NORMAL caches.
     */
    //启用 memcg ，为 KMALLOC_NORMAL 类型缓存池追加 SLAB_NO_MERGE
    if (IS_ENABLED(CONFIG_MEMCG_KMEM) && (type == KMALLOC_NORMAL))
        flags |= SLAB_NO_MERGE;
```

```
    if (minalign > ARCH_KMALLOC_MINALIGN) {
        aligned_size = ALIGN(aligned_size, minalign);
        aligned_idx = __kmalloc_index(aligned_size, false);
    }
    if (!kmalloc_caches[type][aligned_idx])
    //调用 create_kmalloc_cache() 创建缓存池
        kmalloc_caches[type][aligned_idx] = create_kmalloc_cache(
                    kmalloc_info[aligned_idx].name[type],
                    aligned_size, flags);
    if (idx != aligned_idx)
    //原始 idx 与对齐后的 aligned_idx 不同，（尺寸发生了对齐调整）
    //复用缓存池。将 kmalloc_caches[type][idx]
    // 指向 kmalloc_caches[type][aligned_idx];
        kmalloc_caches[type][idx] = kmalloc_caches[type][aligned_idx];
}
```

该函数的参数含义如下：

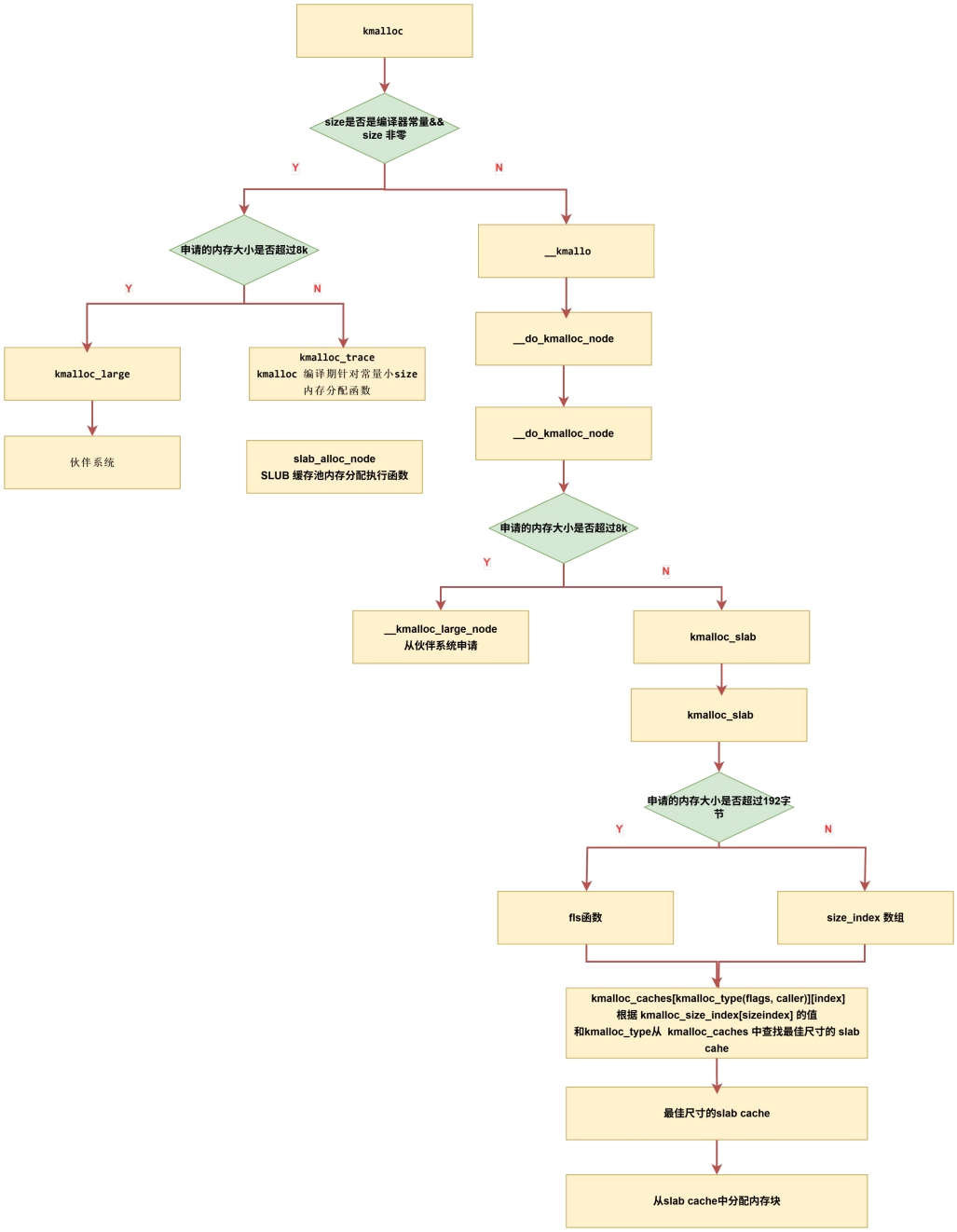
int idx 表示 kmalloc_info[ ] 数组中的 index，对应 slab cache 的尺寸为 2^index 字节，96 字节 和 192 字节这两个尺寸除外，它俩在 kmalloc_info[ ] 数组中的 index 分别为 1 和 2。在 create_kmalloc_caches 函数中会特殊指定。该 idx 也表示 slab cache 在 kmalloc_caches 二维数组中的索引。

enum kmalloc_cache_type type 表示对应的 kmalloc 内存池类型，指定内存来源于哪个物理内存区域，取值范围来自于 enum kmalloc_cache_type 。

slab_flags_t flags 指定创建 slab cache 时的标志位，这里主要用来指定 slab cache 中的内存来源于哪个内存区域。

11.5 kmalloc 内存池内存分配过程

使用 kmalloc 内存池进行内存块分配的过程如下：




```
static __always_inline __alloc_size(1) void *kmalloc(size_t size, gfp_t
flags)
{
    //编译期常量尺寸优化分支
    //GCC 内置函数，检查 size 是否是编译期常量；排除 size == 0
   if (__builtin_constant_p(size) && size) {
```

```
        unsigned int index;
       //编译期发现尺寸超过缓存池上限，直接转发到大尺寸分配路径，
        // 减少了运行时冗余判断
        if (size > KMALLOC_MAX_CACHE_SIZE)
            return kmalloc_large(size, flags);
       //计算缓存池索引。将编译期常量 size 向上对齐到
      //最近的 kmalloc 缓存池尺寸
        index = kmalloc_index(size);
        //从 kmalloc_caches 中匹配最佳缓存池
        return kmalloc_trace(
                kmalloc_caches[kmalloc_type(flags, _RET_IP_)][index],
                flags, size);
    }
    //运行时变量尺寸 ，此路径为通用路径
    return __kmalloc(size, flags);
}
```

```
void *__kmalloc(size_t size, gfp_t flags)
{
    return __do_kmalloc_node(size, flags, NUMA_NO_NODE, _RET_IP_);
}
EXPORT_SYMBOL(__kmalloc);
```

```
/*
 * SLUB directly allocates requests fitting in to an order-1 page
 * (PAGE_SIZE*2).  Larger requests are passed to the page allocator.
 */
#define KMALLOC_SHIFT_HIGH  (PAGE_SHIFT + 1)
/* Maximum size for which we actually use a slab cache */
#define KMALLOC_MAX_CACHE_SIZE  (1UL << KMALLOC_SHIFT_HIGH)

static __always_inline
void *__do_kmalloc_node(size_t size, gfp_t flags, int node,
            unsigned long caller)
{
    struct kmem_cache *s;
    void *ret;
    /*大尺寸分配分支（伙伴系统路径）*/
    // KMALLOC_MAX_CACHE_SIZE 规定 kmalloc 内存池所能管理的内存块最大尺寸
    //，在 slub 实现中是 2 页 大小
```

```
    // 如果使用 kmalloc 申请超过 2 页 大小的内存，则直接走伙伴系统
    if (unlikely(size > KMALLOC_MAX_CACHE_SIZE)) {
    //调用伙伴系统分配大内存。
    //底层调用 alloc_pages 向伙伴系统申请超过 2 页 的内存块
        ret = __kmalloc_large_node(size, flags, node);
        trace_kmalloc(caller, ret, size,
                  PAGE_SIZE << get_order(size), flags, node);
        return ret;
    }
    //size 为零，返回 ZERO_SIZE_PTR
    if (unlikely(!size))
        return ZERO_SIZE_PTR;
    // 根据申请内存块的尺寸 size，在 kmalloc_caches
    //根据申请内存块的尺寸 size，在 kmalloc_caches 缓存中
    //选择合适尺寸的内存池
    s = kmalloc_slab(size, flags, caller);
    //从缓存池中分配内存
    ret = slab_alloc_node(s, NULL, flags, node, caller, size);
    ret = kasan_kmalloc(s, ret, size, flags);
    trace_kmalloc(caller, ret, size, s->size, flags, node);
    return ret;
}
```

当内核向 kmalloc 内存池申请的内存块尺寸 size 超过了 KMALLOC_MAX_CACHE_SIZE 的限制时，内核会绕过 kmalloc 内存池直接到伙伴系统中去申请内存__kmalloc_large_node 函数的调用链底层里边会调用 alloc_pages，随后会进入伙伴系统中申请内存块。

KMALLOC_MAX_CACHE_SIZE 在 slub 的实现中，配置为 8K 大小，也就是说在 slub 的实现中，向 kmalloc 内存池申请的内存块超过了 8K 就会直接走伙伴系统。如果申请的内存块尺寸 size 低于 8k，那么内核就会从 kmalloc_caches 中选取一个最佳尺寸的 slab cache，然后通过这个 slab cache 进行内存块的分配。

从 kmalloc 内存池在 slub 的实现中可以看出，最大能申请的内存块尺寸为 8K，也就是两个物理内存页大小。

11.5.1 从 kmalloc_caches 中选取最佳尺寸的 slab cache

kmalloc 内存池分配内存块的核心是需要在 kmalloc_caches 二维数组中查找到最佳合适尺寸的 slab cache。所以确定了 kmalloc_caches 二维数组的两个维度的索引就确定了最佳尺寸的 slab cache。

```
struct kmem_cache *
kmalloc_caches[NR_KMALLOC_TYPES][KMALLOC_SHIFT_HIGH + 1];
```

11.5.1.2 计算 kmalloc_caches 数组的第一个维度索引的值

kmalloc_caches 数组第一个维度索引值的计算逻辑封装在 kmalloc_type 函数中，在此函数中，会将 kmalloc 接口函数中 gfp_t flags 掩码中指定的物理内存区域转换为 enum kmalloc_cache_type。

```
static __always_inline enum kmalloc_cache_type kmalloc_type(gfp_t flags,
unsigned long caller)
{
    //高频场景单分支检测
    //KMALLOC_NOT_NORMAL_BITS 整合了所有特殊标志位
    //（__GFP_DMA/__GFP_RECLAIMABLE/__GFP_ACCOUNT），
    //通过一次按位与 + 等于 0 的判断，就可以得到所有需要从
    //ZONE_NORMAL 分配内存的场景
    if (likely((flags & KMALLOC_NOT_NORMAL_BITS) == 0))
#ifdef CONFIG_RANDOM_KMALLOC_CACHES
       //随机化缓存池计算。
       // 返回：随机化起点 + 哈希值（范围 0-15）
        return KMALLOC_RANDOM_START + hash_64(caller ^ random_kmalloc_seed,
                                ilog2(RANDOM_KMALLOC_CACHES_NR + 1));
#else
        return KMALLOC_NORMAL;
#endif
   // 特殊标志位优先级匹配
    //优先级 1：__GFP_DMA（硬件强制，最高）。
    //DMA 是外设访问的硬件强制要求，
    //因此优先于 RECLAIMABLE / MEMCG_KME 场景，即使同时设置 __GFP_RECLAIMABLE，
    //也会优先返回 KMALLOC_DMA。
    if (IS_ENABLED(CONFIG_ZONE_DMA) && (flags & __GFP_DMA))
        return KMALLOC_DMA;
   //优先级 2：__GFP_RECLAIMABLE（内存管理，次之）
  //条件中 || 表示：RECLAIMABLE(可回收)是内核内存管理策略，
  //优先级高于 MEMCG_KMEM。
  //即使启用了 memcg，但只要设置了 __GFP_RECLAIMABLE，
  //仍优先返回 KMALLOC_RECLAIM
  //未启用 memcg 时，直接返回 KMALLOC_RECLAIM
    if (!IS_ENABLED(CONFIG_MEMCG_KMEM) || (flags & __GFP_RECLAIMABLE))
        return KMALLOC_RECLAIM;
    Else
       //优先级 3：__GFP_ACCOUNT（MEMCG_KMEM，最低）
         //仅当「启用 memcg + 无 DMA / 可回收标志」时，才返回 KMALLOC_CGROUP
        return KMALLOC_CGROUP;
}
```

kmalloc_type 函数的主要逻辑是，根据 kmalloc 的分配标志 gfp_t flags 和系统配置类型，决定内核向 kmalloc 内存池申请的内存具体属于哪一类的物理内存:

1. 如果标志位 gfp_t flags 中指定了需要从 ZONE_NORMAL 物理内存区域分配内存的场景，并且没有配置 CONFIG_RANDOM_KMALLOC_CACHES 情况下，内核向 kmalloc 内存池申请的内存均来自于 ZONE_NORMAL 物理内存区域。

配置 CONFIG_RANDOM_KMALLOC_CACHES 情况下，内核向 kmalloc 内存池申请的内存来自与随机化缓存池，（随机化缓存池的起始值 + 哈希值（范围 0-15）) 属于 [KMALLOC_RANDOM_START, KMALLOC_RANDOM_END]。

KMALLOC_RANDOM_START 随机化缓存池的起始值：默认等于 KMALLOC_NORMAL，

KMALLOC_RANDOM_END 随机化缓存池的结束值：起始值 + 随机化缓存池数量（默认 15）。

```
#ifdef CONFIG_RANDOM_KMALLOC_CACHES
#define RANDOM_KMALLOC_CACHES_NR    15 // # of cache copies

enum kmalloc_cache_type {
    KMALLOC_NORMAL = 0,
    ......
    KMALLOC_RANDOM_START = KMALLOC_NORMAL,
    KMALLOC_RANDOM_END = KMALLOC_RANDOM_START + RANDOM_KMALLOC_CACHES_NR,
    ......
    NR_KMALLOC_TYPES
};
```

可以看出，KMALLOC_RANDOM_START 永远等于 KMALLOC_NORMAL（无论配置如何），是随机化缓存池的基准起点；

KMALLOC_RANDOM_END = KMALLOC_NORMAL + 15（因为 RANDOM_KMALLOC_CACHES_NR=15），即随机化缓存池的范围是 [KMALLOC_NORMAL, KMALLOC_NORMAL+15]，共 16 个类型。

2. 如果 gfp_t flags 明确指定了 __GFP_DMA，并且 CONFIG_ZONE_DMA 开启的情况下，则内核向 kmalloc 内存池申请的内存均来自于 ZONE_DMA 物理内存区域。

3. 如果 CONFIG_MEMCG_KMEM 开启了，或者 gfp_t flags 明确指定了 __GFP_RECLAIMABLE，内核向 kmalloc 内存池申请的内存均是可以被回收的。

4. 仅 启用 memcg 且无 DMA / 可回收标志时，申请的内存类型是 KMALLOC_CGROUP。

11.5.1.2 计算 kmalloc_caches 数组的第二个维度索引的值

如何找到这个最佳的 slab cache 在 kmalloc_caches 中的索引 index 呢？

```
/*
 * Find the kmem_cache structure that serves a given size of
 * allocation
 *
```

```
 * This assumes size is larger than zero and not larger than
 * KMALLOC_MAX_CACHE_SIZE and the caller must check that.
 */
static inline struct kmem_cache *
kmalloc_slab(size_t size, gfp_t flags, unsigned long caller)
{
    unsigned int index;
    // 如果申请的内存块 size 在 192 字节以下，
    //则通过 kmalloc_size_index 数组定位 kmalloc_caches 缓存索引
    // 从而获取到最佳合适尺寸的内存池 slab cache
    if (size <= 192)
        index = kmalloc_size_index[size_index_elem(size)];
    else
      // 如果申请的内存块 size 超过 192 字节
      //，则通过 fls 定位 kmalloc_caches 缓存的 index
     // fls 可以获取参数的最高有效 bit 的位数
      //，比如 fls(0)=0，fls(1)=1，fls(4) = 3
     index = fls(size - 1);
    // 根据 kmalloc_type 以及 index 获取最佳尺寸的内存池 slab cache
    return kmalloc_caches[kmalloc_type(flags, caller)][index];
}
```

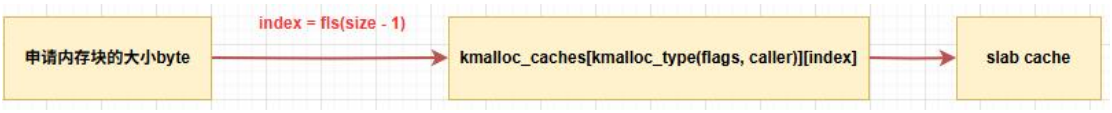
当申请内存块的尺寸小于 192 字节时，内核会通过 kmalloc_size_index 数组来定位 kmalloc_caches 中 slab cache 的 index。kmalloc_size_index 数组中存放的值正是 kmalloc_caches 中的索引 index。

```
/*
 * Conversion table for small slabs sizes / 8 to the index in the
 * kmalloc array. This is necessary for slabs < 192 since we have non power
 * of two cache sizes there. The size of larger slabs can be determined using
 * fls.
 */
u8 kmalloc_size_index[24] __ro_after_init = {
    3,  /* 8 */
    4,  /* 16 */
    5,  /* 24 */
    5,  /* 32 */
    6,  /* 40 */
    6,  /* 48 */
    6,  /* 56 */
    6,  /* 64 */
```

```
    1,  /* 72 */
    1,  /* 80 */
    1,  /* 88 */
    1,  /* 96 */
    7,  /* 104 */
    7,  /* 112 */
    7,  /* 120 */
    7,  /* 128 */
    2,  /* 136 */
    2,  /* 144 */
    2,  /* 152 */
    2,  /* 160 */
    2,  /* 168 */
    2,  /* 176 */
    2,  /* 184 */
    2   /* 192 */
};
```

如果能通过申请内存块的大小 size，定位到 kmalloc_size_index 数组的索引 size_index，那么就可以通过 kmalloc_size_index[size_index] 找到 kmalloc_caches 中的最佳 slab cache 。在内核中通过 size_index_elem 函数来根据申请内存块的尺寸 bytes，计算 kmalloc_size_index 数组的索引 size_index。然后根据 kmalloc_size_index[size_index] 的值以及 gfp_t flags 中指定的 kmalloc_type 从 kmalloc_caches 中直接查找出最佳合适尺寸的 slab cahe 出来。

```
static inline unsigned int size_index_elem(unsigned int bytes)
{
    return (bytes - 1) / 8;
}
```



上述为 二维数组 kmalloc_caches 的第一维索引 index 的获取逻辑，分为两种情况：

1. 申请内存块的尺寸 < 192 字节
2. 申请内存块的尺寸 >= 192 字节

```
struct kmem_cache *
kmalloc_caches[NR_KMALLOC_TYPES][KMALLOC_SHIFT_HIGH + 1];
```

11.6 kmalloc 内存池对内存回收

kfree 函数来释放由 kmalloc 内存池分配的内存块，参数 object 表示释放内存块的虚拟内存地址。

```
/**
 * kfree - free previously allocated memory
 * @object: pointer returned by kmalloc() or kmem_cache_alloc()
 *
 * If @object is NULL, no operation is performed.
 */
void kfree(const void *object)
{
    struct folio *folio;
    struct slab *slab;
    //对象所属的 kmem_cache 缓存池
    struct kmem_cache *s;
    // object 为要释放的内存块的虚拟内存地址
    void *x = (void *)object;

    trace_kfree(_RET_IP_, object);
    if (unlikely(ZERO_OR_NULL_PTR(object)))
        return;
    //struct folio: : 管理「单页 / 连续多页」内存块，替代旧版的 page
   // 通过虚拟内存地址找到内存块所属的 folio 的地址
    folio = virt_to_folio(object);
    //判断是否为大对象（非 SLUB 缓存池分配），大对象由伙伴系统分配，
    //需走专属释放逻辑
    if (unlikely(!folio_test_slab(folio))) {
         //释放大对象（归还给伙伴系统）
        free_large_kmalloc(folio, (void *)object);
        return;
    }
    //获取对象所属的 slab 和缓存池
    slab = folio_slab(folio);
    //获取对象所属缓存池
    s = slab->slab_cache;
    //将内存块释放回其所在的 slub 中
    slab_free(s, slab, x, _RET_IP_);
}
EXPORT_SYMBOL(kfree);
```

首先内核需要通过 virt_to_folio 函数，根据内存块的虚拟内存地址 x 找到其所在的 folio 内存块的首地址。

再通过 folio_test_slab 函数判断该 folio 是否被 SLUB 缓存池管理，如果不被 SLUB 缓存

池管理，则待释放对象是大对象，是由伙伴系统分配的，调用 free_large_kmalloc 释放；如果被 SLUB 缓存池管理，则待释放对象是小对象，后续调用 slab_free 释放。